

プログラミング及び実習2

第 1 回 イントロ、C プログラミング環境の構築、シェルコマンドの操作

佐野 彰

本日の内容

1. 前期科目との違い (10 分)

- 独自環境 (タートルグラフィックス) → 各 OS 標準の C 言語環境へ

2. C 言語プログラミング環境の構築 (30 分)

- Windows (WSL)、macOS それぞれの構築方法

3. ターミナル (CUI) の操作 (15 分)

- シェルコマンド、WebTerm Learn での練習

4. 演習課題の進め方 (5 分)

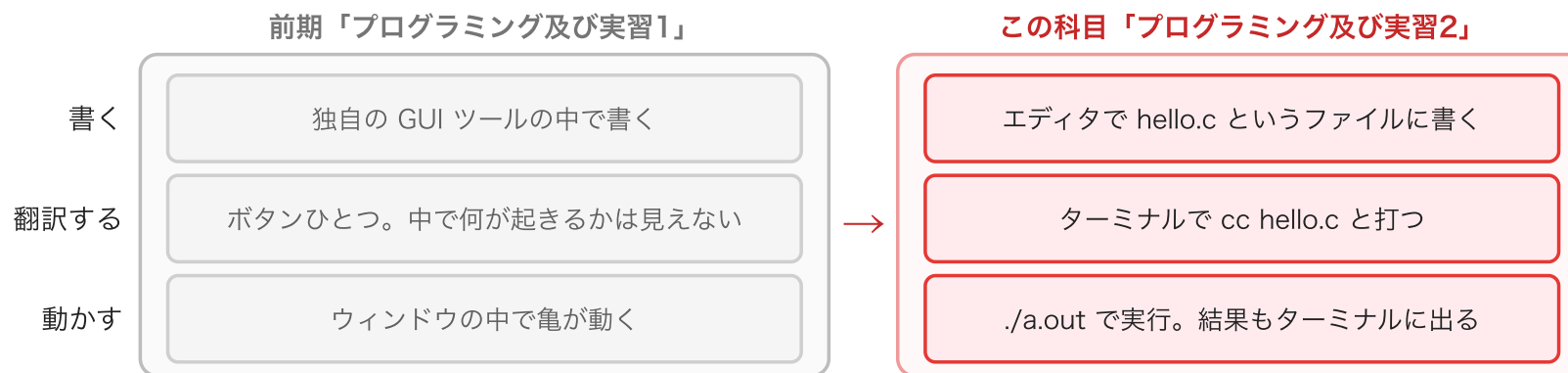
- ex01-1・ex01-2・ex01-3、aiJudge の使い方

1. 前期科目との違い

タートルグラフィックスの独自環境から標準環境へ

前期「プログラミング及び実習1」では、初学者の理解を促すため、タートルグラフィックスを利用した数理課程の独自環境で C 言語プログラミングを体験しました。

この科目では、2 年次以降の科目でも利用する各 OS 標準の方法で、より一般的な C 言語プログラミング環境を各自の PC 上に構築します。



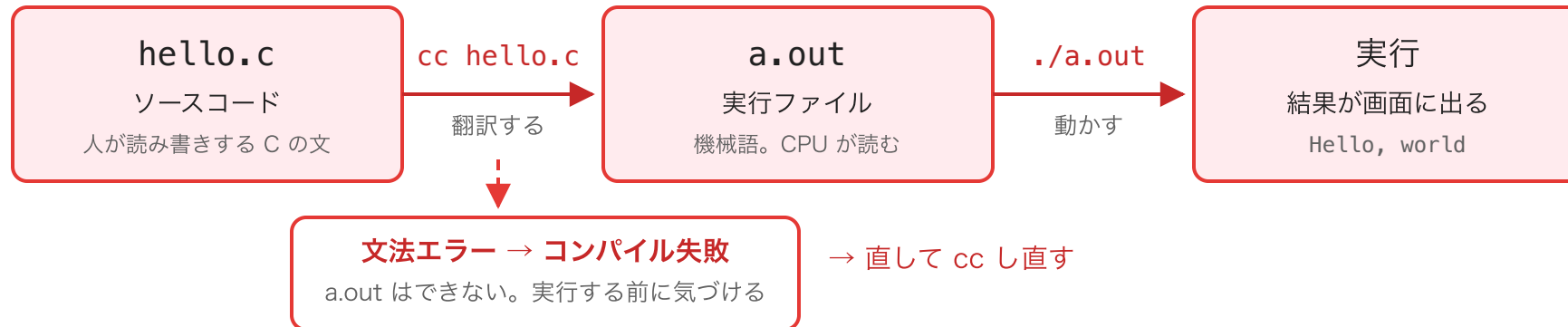
道具が変わるだけではない。どの段階で何が起きているかが、自分から見えるようになる。

- 独自の GUI ツールではなく、**ターミナル (CUI) + 標準コンパイラ**を直接使う
- Windows なら **WSL (Ubuntu)**、macOS なら **標準の開発者ツール**
- ここで身につける環境操作は、ネットワークやデータ分析などの科目でもそのまま使う

2. C 言語プログラミング環境の構築

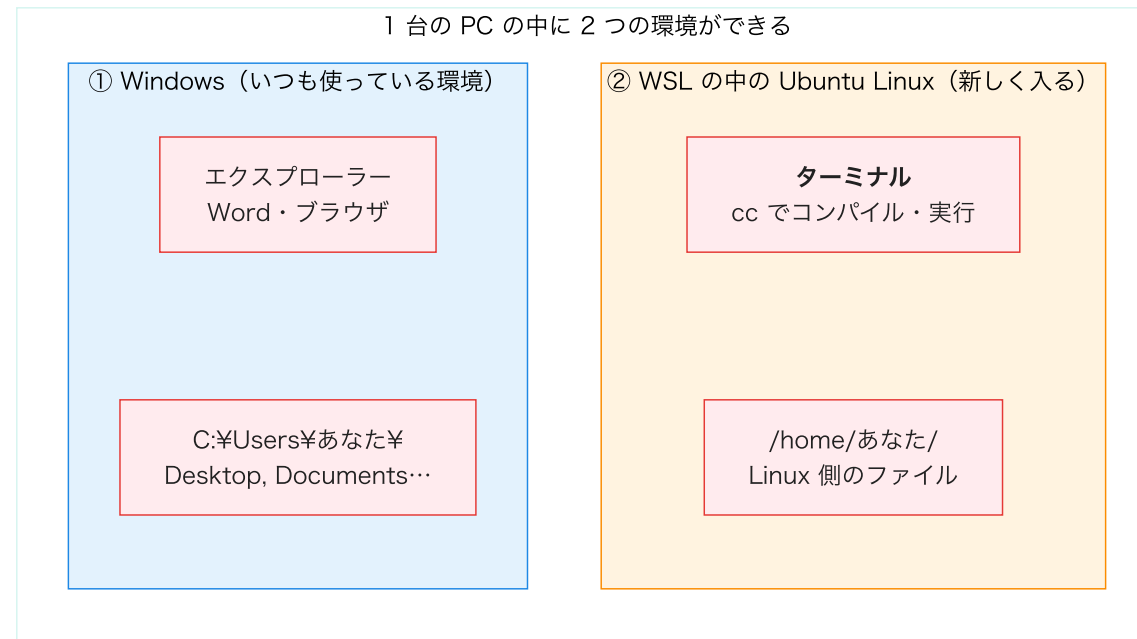
コンパイルから実行までの流れ

C 言語はコンパイラ言語です。ソースコードを機械語に翻訳（コンパイル）してから実行します。



- `cc` コマンドがコンパイラを起動する
- コンパイルが通ってはじめて実行ファイル（`a.out`）ができる
- 文法エラーはこの時点で見つかる（実行前に気づける）

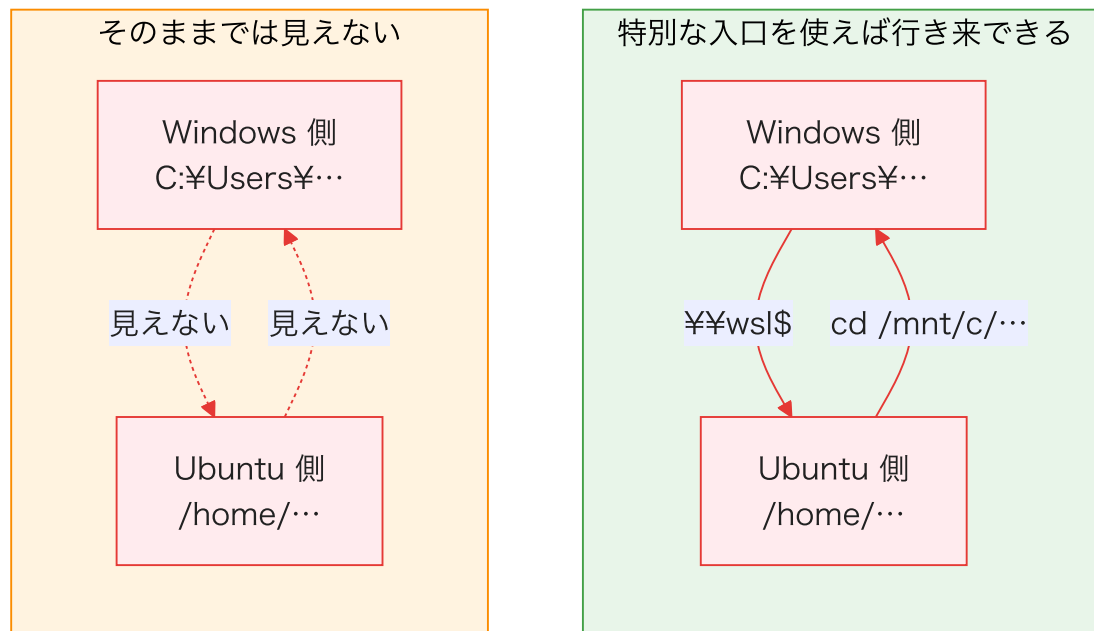
Windows の人へ — WSL とは何か



- C 言語の開発環境は **Linux** 上に作る。Windows に Linux を入れる仕組みが **WSL** (Windows Subsystem for Linux)
- PC は 1 台のままだが、中に 2 つの環境ができる。C 言語の作業 (cc でコンパイル・実行) はすべて **Ubuntu 側のターミナル**で行う

macOS はもともと UNIX 系なので、この仕組みは不要。ターミナルをそのまま使う。

WSL の落とし穴 — ファイルが「見えない」



2 つの環境はファイルの置き場所も別

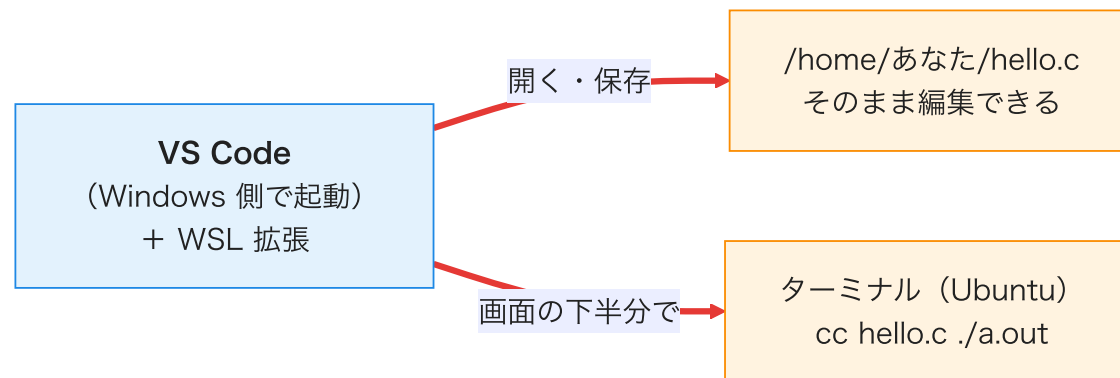
- Ubuntu のターミナルで `ls` しても、Windows のデスクトップにあるファイルは出てこない
- 逆にエクスプローラーで探しても、Ubuntu で作った `hello.c` は見つからない

行き来するには入口を使う

- Ubuntu から Windows を見る
 - `cd /mnt/c/Users/あなた/Desktop`
- Windows から Ubuntu を見る
 - エクスプローラーのアドレス欄に `\\wsl$`

迷ったら「今どちらにいるか」を確認する。プロンプトが `あなた@PC名:~$` なら Ubuntu 側。
この科目の課題は Ubuntu 側に置いて、Ubuntu 側でコンパイルする。

おすすめ — VS Code を WSL につないで使う



- VS Code に WSL 拡張を入れると、Windows 上の VS Code から Ubuntu 側のファイルを直接開いて編集できる
 - `/mnt/c/...` や `\\wsl$` を意識しなくてよくなる。前ページの「行き来」で迷わない
- ターミナルも VS Code の中に開ける。編集とコンパイルが 1 つの画面で完結する
- 左下の緑色の表示が **WSL: Ubuntu** になっていれば、Ubuntu 側につながっている

手順は公開ページの「WSL のための Visual Studio Code 設定」。
Windows で WSL を使う人には強く推奨します。

環境構築の方法

Windows

1. Windows への WSL の導入と日本語化
2. Ubuntu Linux の C 言語開発環境

macOS

1. macOS の C 言語開発環境
 - 手順は[公開ページ](#)の「環境構築」から辿れます
 - 構築できたら [簡単な C プログラムをコンパイル・実行してみる](#) で動作確認

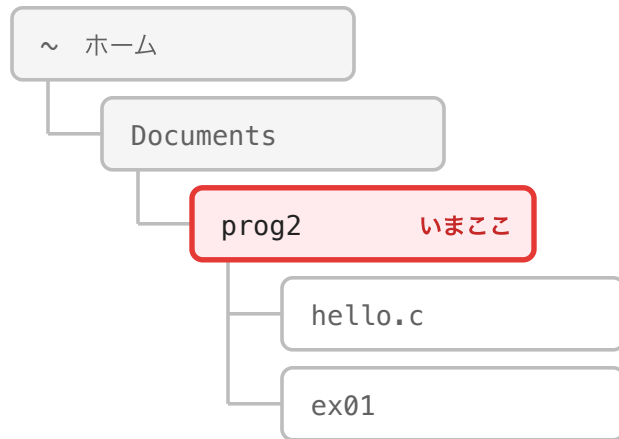
Windows の人は、前ページの **VS Code** の **WSL 連携** もあわせて設定しておくと作業が楽になります。
手順は公開ページの「環境構築」から。

3. ターミナル (CUI) の操作

シェルコマンドに慣れる

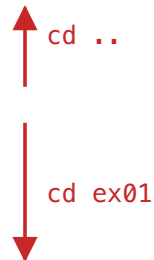
この科目では、ターミナル（端末）と呼ばれるコマンドベースのインターフェイス（CUI）を用いて、ファイルの操作や C プログラムのコンパイル・実行などを行います。

ファイルの階層



コマンドは「いまいる場所」を基準に働く

<code>pwd</code>	いまいる場所を表示する /home/you/Documents/prog2
<code>ls</code>	そこにあるものを並べる hello.c ex01
<code>mkdir ex02</code>	そこに新しいフォルダを作る
<code>rm hello.c</code>	そこにあるファイルを消す ごみ箱には入らない。戻せない



GUI でフォルダを開いて中を見ているのと同じこと。違うのは、場所とものを「名前で指す」点だけ。

- `cd`、`ls`、`mkdir`、`rm` などの基本コマンドを、実際に使いながら少しずつ覚えていく
- GUI（Finder / エクスプローラー）のできることの多くは、CUI でもできる
- 参考: Unix/Linux コマンドリファレンス（日本語チートシート）

WebTerm Learn でターミナル操作を練習する

[WebTerm Learn](#) は、ブラウザだけでターミナル操作を学べる無料の学習サイトです。インストール不要で、ゲームのように「冒険マップ」を進みながらレッスンを進めます。

- Google または GitHub アカウントでサインイン
- 「根の森 — ターミナル入門編」(全 20 レッスン) に取り組む
- 修了すると修了証 (PNG) をダウンロードできる

詳しくは公開ページの [WebTerm Learn でターミナル操作を練習する](#) を参照。

4. 演習課題の進め方

演習課題 ex01

ex01-1: C 言語環境の構築 (必須)

各 OS で環境構築 → `cc` コマンドを実行したターミナルのスクリーンショットを撮影 → aiJudge に提出

ex01-1': より快適な環境設定 (任意・推奨)

VS Code のインストール、WSL 連携の設定など

ex01-2: WebTerm Learn (必須)

「根の森 — ターミナル入門編」を修了 → 修了証 (PNG) を aiJudge に提出

ex01-3: paiza ラーニング「Linux 入門編」(任意)

修了した各レッスンの認定証を aiJudge に提出

課題の提出には aiJudge を使います

この科目では、課題の提示・提出をすべて [aiJudge](#)（オンラインジャッジ）で行います。プログラムだけでなく、画像・PDF の提出も aiJudge で受け付けます（Teams には提出しません）。

1. [aiJudge](#) を開き、龍谷大学の全学認証（大学アカウント）でログイン
 - 。個別の ID・パスワードの配布はありません
2. 使い方は [aiJudge 学生向けガイド](#) を参照してください。

画像・PDF は自動採点ではありません。プログラムの課題は提出直後に自動採点されますが、画像・PDF の提出は教員・TA が確認してから採点します。

次回予告と今日やること

次回（10/1）：C プログラムのコンパイル、C プログラミングの復習

- ジャッジへの初回提出（プログラム課題）
- 前期の C 言語の復習

今日の 5 講時（実習）

1. 各自の OS に合わせて C 言語環境を構築
2. `cc` コマンドで簡単なプログラムをコンパイル・実行
3. aiJudge にログインできることを確認
4. WebTerm Learn・paiza ラーニングの課題に着手

質問は Teams 「★ 質問用チャネル」 か、教室の TA へ。